Lab 07: The Impact of Data Scale and Indexing on Query Performance

Name: Abdullah Ayub
Roll No: BSDSF24M029

1. Objective

This lab measures how query execution time changes as a university database grows from 1,000 to 1,000,000 students, and how much indexing improves performance at the largest scale. Five queries of increasing complexity were timed at four data scales without indexes, then re-timed at 1 million students after adding indexes. The database is Microsoft SQL Server (Developer Edition), data was generated with Python (Faker and pyodbc), and times were captured with SET STATISTICS TIME ON.

2. SQL Schema

The database contains five tables: Departments, Teachers, Courses, Students and Enrollments (a junction table between students and courses).

```
CREATE DATABASE university_db;
GO

USE university_db;
GO

CREATE TABLE Departments (
    department_id INT IDENTITY(1,1) PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL,
    building VARCHAR(100)
);

CREATE TABLE Teachers (
    teacher_id INT IDENTITY(1,1) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    department_id INT,
    hire_date DATE,
    FOREIGN KEY (department_id)
        REFERENCES Departments(department_id)
);

CREATE TABLE Courses (
    course_id INT IDENTITY(1,1) PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT NOT NULL,
    teacher_id INT,
    FOREIGN KEY (teacher_id)
        REFERENCES Teachers(teacher_id)
);

CREATE TABLE Students (
    student_id INT IDENTITY(1,1) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    enrollment_date DATE,
    date_of_birth DATE
);

CREATE TABLE Enrollments (
    enrollment_id INT IDENTITY(1,1) PRIMARY KEY,
    student_id INT NOT NULL,
```

```
    course_id INT NOT NULL,
    semester VARCHAR(30) NOT NULL,
    grade INT CHECK (grade BETWEEN 0 AND 100),
    FOREIGN KEY (student_id)
        REFERENCES Students(student_id),
    FOREIGN KEY (course_id)
        REFERENCES Courses(course_id)
);
GO
```

3. Data Generation Script

The script creates 10 departments, 100 teachers and 200 courses, then inserts students in batches of 5,000 and enrolls each student in 5-10 random courses (grades 50-100, four semesters). It was run once per scale by editing the call in the main block; the scale 4 call is the one left active below.

```
import random
from datetime import date
from faker import Faker
import pyodbc

# Initialize Faker
fake = Faker()

# Database connection configuration for SQL Server Developer Edition
CONN_STR = (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    "SERVER=localhost;"  # Update server name if necessary
    "DATABASE=IndexDB;"
    "Trusted_connection=yes;"
)



def generate_university_data(num_students: int):
  conn = pyodbc.connect(CONN_STR)
  cursor = conn.cursor()

  # Enable fast_executemany for high-speed bulk inserts in pyodbc
  cursor.fast_executemany = True

  print("\n" + "=" * 50)
  print(f"Starting Data Generation for Scale: {num_students:,} Students")
  print("=" * 50)

  # 1. Insert 10 Departments
  print("Inserting 10 Departments...")
  departments = [
      ("Computer Science", "Block A"),
      ("Physics", "Science Wing"),
      ("Mathematics", "Block B"),
      ("Electrical Engineering", "Engineering Hub"),
      ("Mechanical Engineering", "Engineering Hub"),
      ("Business Administration", "Management Block"),
      ("Economics", "Management Block"),
      ("Chemistry", "Science Wing"),
      ("Biology", "Science Wing"),
      ("English Literature", "Arts Block"),
  ]
  cursor.executemany(
      """
        INSERT INTO Departments (department_name, building)
        VALUES (?, ?)
    """,
      departments,
  )
  conn.commit()

  # 2. Insert 100 Teachers
  print("Inserting 100 Teachers...")
  teachers = []
```

```
for _ in range(100):
  first_name = fake.first_name()
  last_name = fake.last_name()
  email = fake.unique.email()
  dept_id = random.randint(1, 10)
  hire_date = fake.date_between(start_date="-10y", end_date="today")
  teachers.append((first_name, last_name, email, dept_id, hire_date))

cursor.executemany(
    """
      INSERT INTO Teachers (first_name, last_name, email, department_id, hire_date)
      VALUES (?, ?, ?, ?, ?)
  """,
    teachers,
)
conn.commit()

# 3. Insert 200 Courses
print("Inserting 200 Courses...")
base_courses = [
    "Introduction to Algorithms",
    "Quantum Mechanics",
    "Linear Algebra",
    "Circuit Analysis",
    "Thermodynamics",
    "Financial Accounting",
    "Microeconomics",
    "Organic Chemistry",
    "Cell Biology",
    "Shakespearean Drama",
]
courses = []
for i in range(200):
  # Ensure unique course names by appending an index if needed
  base_name = base_courses[i % len(base_courses)]
  c_name = f"{base_name} {i//len(base_courses) + 1}" if i >= 10 else base_name
  credits = random.choice([3, 4])
  teacher_id = random.randint(1, 100)
  courses.append((c_name, credits, teacher_id))

cursor.executemany(
    """
      INSERT INTO Courses (course_name, credits, teacher_id)
      VALUES (?, ?, ?)
  """,
    courses,
)
conn.commit()

# 4. Insert Students and Enrollments in Batches
print(
    f"Inserting {num_students:,} Students and corresponding Enrollments..."
)

semesters = ["Fall 2023", "Spring 2024", "Fall 2024", "Spring 2025"]
batch_size = 5000  # Batch size to manage memory and transaction logs

# To link enrollments correctly with auto-incrementing student IDs,
# we insert students in chunks, fetch their assigned IDs, and insert enrollments.
total_enrollments = 0

for start_idx in range(1, num_students + 1, batch_size):
  end_idx = min(start_idx + batch_size - 1, num_students)
  current_batch_size = end_idx - start_idx + 1

  student_batch = []
  for _ in range(current_batch_size):
    first_name = fake.first_name()
    last_name = fake.last_name()
    email = fake.unique.email()
    enrollment_date = fake.date_between(start_date="-4y", end_date="today")
    dob = fake.date_of_birth(minimum_age=18, maximum_age=30)
    student_batch.append((first_name, last_name, email, enrollment_date, dob))
```

```
    # Insert students
    cursor.executemany(
        """
            INSERT INTO Students (first_name, last_name, email, enrollment_date, date_of_birth)
            VALUES (?, ?, ?, ?, ?)
        """,
        student_batch,
    )
    conn.commit()

    # Retrieve the student IDs just generated for this batch
    cursor.execute(
        """
        SELECT TOP {} student_id FROM Students ORDER BY student_id DESC
    """.format(
            current_batch_size
        )
    )
    # Fetch them in the correct ascending order
    batch_student_ids = [row[0] for row in cursor.fetchall()][::-1]

    # Generate enrollments (each student takes 5 to 10 random courses)
    enrollment_batch = []
    for s_id in batch_student_ids:
      num_courses = random.randint(5, 10)
      # Pick random unique courses for the student
      enrolled_courses = random.sample(range(1, 201), num_courses)
      for c_id in enrolled_courses:
        semester = random.choice(semesters)
        grade = random.randint(50, 100)  # Realistic passing/grading range
        enrollment_batch.append((s_id, c_id, semester, grade))

    cursor.executemany(
        """
            INSERT INTO Enrollments (student_id, course_id, semester, grade)
            VALUES (?, ?, ?, ?)
        """,
        enrollment_batch,
    )
    conn.commit()
    total_enrollments += len(enrollment_batch)
    print(
        f"Progress: Processed up to student {end_idx:,} / {num_students:,}"
        f" ({total_enrollments:,} total enrollments so far)"
    )

  cursor.close()
  conn.close()
  print("Data generation completed successfully!")



def clear_database():
  conn = pyodbc.connect(CONN_STR)
  cursor = conn.cursor()
  print("\nClearing existing data and resetting identity seeds...")

  # Delete in correct child-to-parent order
  cursor.execute("DELETE FROM Enrollments")
  cursor.execute("DELETE FROM Students")
  cursor.execute("DELETE FROM Courses")
  cursor.execute("DELETE FROM Teachers")
  cursor.execute("DELETE FROM Departments")

  # Reset the identity seeds back to 1 for all tables
  cursor.execute("DBCC CHECKIDENT ('Enrollments', RESEED, 0)")
  cursor.execute("DBCC CHECKIDENT ('Students', RESEED, 0)")
  cursor.execute("DBCC CHECKIDENT ('Courses', RESEED, 0)")
  cursor.execute("DBCC CHECKIDENT ('Teachers', RESEED, 0)")
  cursor.execute("DBCC CHECKIDENT ('Departments', RESEED, 0)")

  conn.commit()
  cursor.close()
```

```
  conn.close()
  print("Database cleared and identity seeds reset.")

if __name__ == "__main__":
  # Run for Scale 1 (1,000 students)
  #generate_university_data(1000)

  # Scale 2 (10,000 students):
   #clear_database()
   #generate_university_data(10000)

  # Scale 3 (100,000 students):
  #clear_database()
  #generate_university_data(100000)

  # Scale 4 (1,000,000 students):
    clear_database()
    generate_university_data(1000000)
```

4. Queries and Indexes

4.1 Queries

```
USE IndexDB;
GO
SET STATISTICS TIME ON;
GO

-- Query 1: Simple filter
SELECT student_id, first_name, last_name, email, enrollment_date, date_of_birth
FROM Students
WHERE YEAR(enrollment_date) = 2023;

-- Query 2: Simple join and filter
SELECT DISTINCT s.email
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE c.teacher_id = 50;

-- Query 3: Multi-join with text search
SELECT DISTINCT t.first_name, t.last_name
FROM Teachers t
JOIN Courses c ON t.teacher_id = c.teacher_id
WHERE c.course_name LIKE '%Advanced%';

-- Query 4: Join with aggregation
SELECT d.department_name, COUNT(c.course_id) AS total_courses
FROM Departments d
LEFT JOIN Teachers t ON d.department_id = t.department_id
LEFT JOIN Courses c ON t.teacher_id = c.teacher_id
GROUP BY d.department_id, d.department_name;

-- Query 5: Complex join, aggregation, filter, sort, top 10
SELECT TOP 10
    s.first_name,
    s.last_name,
    AVG(CAST(e.grade AS FLOAT)) AS average_grade
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.semester = 'Spring 2025'
GROUP BY s.student_id, s.first_name, s.last_name
ORDER BY average_grade DESC;

SET STATISTICS TIME OFF;
GO
```

4.2 Indexes (created before the post-indexing run)

```
USE IndexDB;
```

```
GO

CREATE NONCLUSTERED INDEX IX_Students_EnrollmentDate
ON Students(enrollment_date);
GO

CREATE NONCLUSTERED INDEX IX_Enrollments_StudentId
ON Enrollments(student_id);
GO

CREATE NONCLUSTERED INDEX IX_Enrollments_CourseId
ON Enrollments(course_id);
GO

CREATE NONCLUSTERED INDEX IX Enrollments Semester
ON Enrollments(semester);
GO

CREATE NONCLUSTERED INDEX IX_Courses_TeacherId
ON Courses(teacher_id);
GO

CREATE NONCLUSTERED INDEX IX_Courses_CourseName
ON Courses(course_name);
GO

CREATE NONCLUSTERED INDEX IX_Teachers_DepartmentId
ON Teachers(department_id);
GO
```

5. Timing Data

Execution time in milliseconds, as reported by SET STATISTICS TIME.

5.1 Without indexes, by scale

Query	1k	10k	100k	1M
Q1: Simple filter	1	130	349	2569
Q2: Join and filter	18	36	517	744
Q3: Multi-join, LIKE	0	0	0	0
Q4: Join with GROUP BY	5	6	3	3
Q5: Complex join, AVG, top 10	4	42	292	1874

5.2 Rows returned at each scale

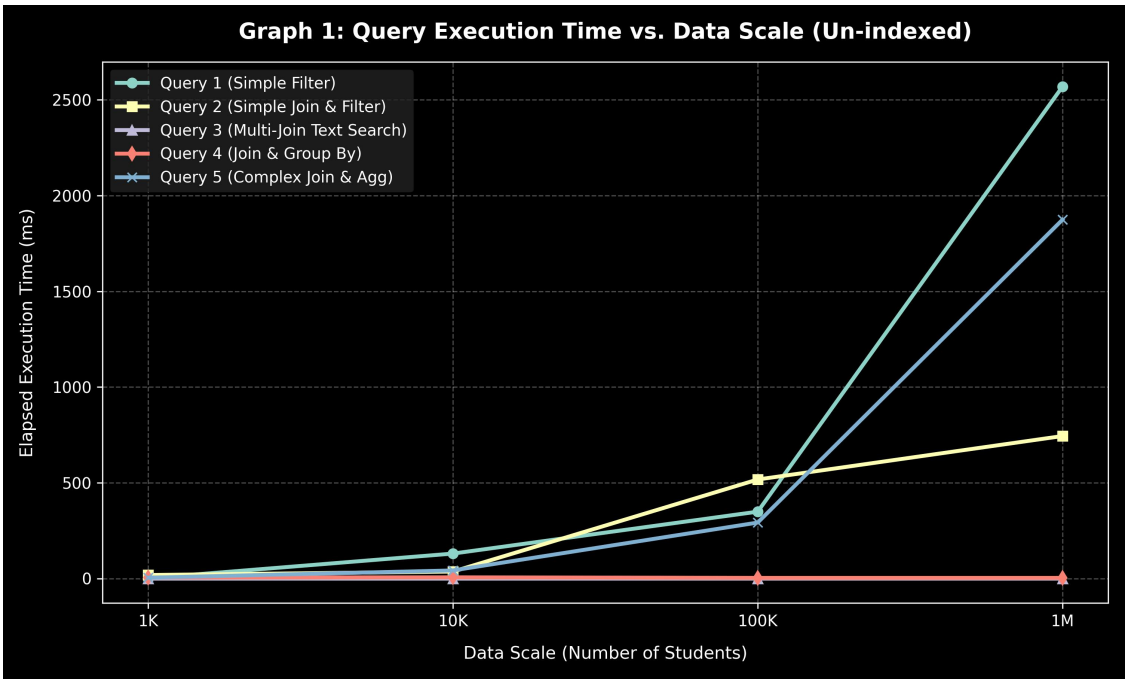
Query	1k	10k	100k	1M
Q1	247	2,451	25,031	250,139
Q2	31	410	20,713	37,506
Q3	0	0	0	0
Q4	10	10	10	10
Q5	10	10	10	10

5.3 Before vs. after indexing (1M students)

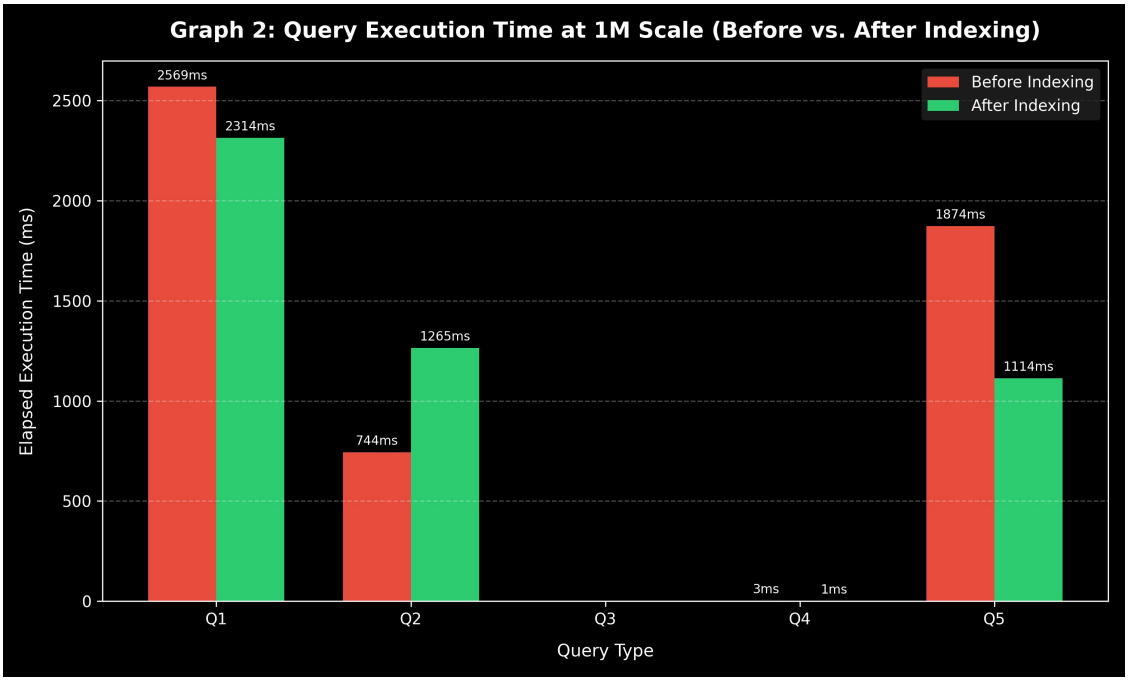
Query	Before (ms)	After (ms)	Change
Q1: Simple filter	2569	2314	9.9% faster
Q2: Join and filter	744	1265	70.0% slower
Q3: Multi-join, LIKE	0	0	No change
Q4: Join with GROUP BY	3	1	66.7% faster*
Q5: Complex join, AVG, top 10	1874	1114	40.6% faster

* Q4 differs by only 2 ms, which is close to the timer's resolution.

6. Graphs



Graph 1: Query execution time vs. data scale (un-indexed)



Graph 2: Execution time at 1M scale, before vs. after indexing

7. Conclusion

Which query was most affected by the increase in data volume?

Query 1 grew the most in absolute terms, from 1 ms to 2,569 ms, followed by Query 5, from 4 ms to 1,874 ms. Query 1 has no usable index, so it must read every row of the Students table, and its cost grows with the table size (it also returns 250,139 rows at 1M). Query 5 has to scan and join the Enrollments table, which holds millions of rows at 1M students, then filter by semester, group by student, average the grades and sort. Each of those steps scales with the size of Enrollments, so it is the most expensive operation overall. Query 2 also grew noticeably (18 ms to 744 ms), but part of that depends on how many courses teacher 50 happens to teach in each generated dataset, not on scale alone.

Queries 3 and 4 stayed flat because they only touch the small Teachers, Courses and Departments tables, whose sizes (100, 200 and 10 rows) do not change with the number of students.

Which query saw the most significant improvement after indexing?

Query 5 improved the most in a meaningful way: 1,874 ms down to 1,114 ms (40.6%, about 760 ms saved). The indexes on Enrollments(semester) and Enrollments(student_id) let SQL Server find the Spring 2025 rows and match them to students without scanning the entire Enrollments table. Query 4 shows a larger percentage (66.7%), but 3 ms to 1 ms is within measurement noise, so it should not be treated as a real gain.

Was there any query that did not improve much? Why?

- **Query 1 (only 9.9% faster).** The filter is WHERE YEAR(enrollment_date) = 2023. Wrapping the column in a function prevents SQL Server from doing an index seek on IX_Students_EnrollmentDate, because the function has to be evaluated for every row. In addition, the query returns about 250,000 of 1,000,000 rows and selects columns that the index does not contain, so reading the table directly is cheaper than looking up each row from the index. Rewriting the filter as a range (enrollment_date >= '2023-01-01' AND enrollment_date < '2024-01-01') would let the index be used for a seek; this was not tested here.

- **Query 2 (got slower, 744 ms to 1,265 ms).** It returns 37,506 rows after joining three tables. With the new indexes available, the optimizer may have chosen a plan that does many index lookups, which can cost more than a scan with a hash join when a large number of rows is involved. Caching between runs could also contribute. Comparing the execution plans before and after would confirm the cause.
- **Query 3 (0 ms before and after).** It returned 0 rows because no course name contains the word 'Advanced': the generated names are the ten base titles followed by a number. There was therefore nothing to measure. Even with matching data, LIKE '%Advanced%' starts with a wildcard, which an index cannot seek on, and the Courses table has only 200 rows.
- **Query 4.** It already ran in about 3 ms because it only touches three tiny tables. There was nothing left to speed up.

What are the downsides of adding too many indexes?

- **Slower writes.** Every INSERT must also add an entry to each index on the table, every DELETE must remove it, and every UPDATE of an indexed column must update the index. With many indexes on a large table such as Enrollments, bulk loading and frequent updates become noticeably slower.
- **More storage.** Each index is a separate structure stored on disk (and cached in memory), which can add up to a large fraction of the table's own size.
- **Maintenance and planning overhead.** Indexes need to be maintained (for example against fragmentation), and having many candidate indexes gives the optimizer more plans to consider and can lead to poor choices, as possibly seen in Query 2.

In practice, indexes should be added for columns used often in selective WHERE, JOIN and ORDER BY clauses and verified with the execution plan, rather than added on every column.